

Report conversation

I'm having a problem with my GitHub Action and my deploy script. Can you help with that?

Absolutely, I'd be happy to help you with your GitHub Action and deploy script. Could you please provide more details about the problem you're experiencing? Information such as error messages, logs, or descriptions of the behavior you're encountering can be very useful in diagnosing and fixing the issue.

So this is my step to deploy a new alpha version:

yaml

```
- name: Deploy iOS alpha to TestFlight
  env:
    # The following secrets are used here
    # to use the same names as the CLI
    CERTIFICATE_PRIVATE_KEY: ${ secrets.CERTIFICATE_PRIVATE_KEY }
    APP_STORE_CONNECT_KEY_IDENTIFIER: ${ secrets.APP_STORE_CONNECT_KEY_IDENTIFIER }
    APP_STORE_CONNECT_ISSUER_ID: ${ secrets.APP_STORE_CONNECT_ISSUER_ID }
    APP_STORE_CONNECT_PRIVATE_KEY: ${ secrets.APP_STORE_CONNECT_PRIVATE_KEY }
  run: |
    # Because we are publishing every build
    # we are able to use the last commit message
    # as a note for the alpha builds. This
    # is much better than nothing.
    #
    # The "sed 's/[<>]//g'" part is to
    # remove any special characters
    # because the App Store doesn't allow
    # special characters in the release notes.
    export LAST_COMMIT_MESSAGE=$(git log --format=%B -n 1 | sed 's/[<>]//g')
    export SHORT_LAST_COMMIT_MESSAGE=$(echo $LAST_COMMIT_MESSAGE | fold -s 4000 | tr -d '\n')
    # The App Store Connect API only allows
    # a maximum of 4000 characters for
    # the release notes. So we need to
    # truncate the message if it is
    # longer than 4000 characters.
    export SHORT_LAST_COMMIT_MESSAGE=$(echo $LAST_COMMIT_MESSAGE | fold -s 4000 | tr -d '\n')
```

```
# Tempoary prints
echo "LAST_COMMIT_MESSAGE: $LAST_C
echo "SHORT_LAST_COMMIT_MESSAGE: $

sz deploy ios \
  --stage alpha \
  --whats-new="$SHORT_LAST_COMMIT_
  --export-options-plist=$HOME/exp
```

As you can see, I pass the last commit message as "whats-new" parameter. The `sz deploy ios` command is my own CLI that I programmed written in Dart. This is the relevant code for the CLI:

```
dart
class DeployIosCommand extends Command {
  final SharezoneRepo _repo;

  DeployIosCommand(this._repo) {
    argParser
      ..addOption(
        releaseStageOptionName,
        abbr: 's',
        allowed: _iosStages,
        help:
          'The deployment stage to deploy
          defaultsTo: 'stable',
      )
      ..addOption(
        exportOptionsPlistName,
        help:
          'Export an IPA with these option
      )
      ..addOption(
        flavorOptionName,
        allowed: _iosFlavors,
        help: 'The flavor to build for. Only
        defaultsTo: 'prod',
      );
  }
}
```

```

        addAppStoreConnectKeyIdOption(argParser);
        addAppStoreConnectIssuerIdOption(argParser);
        addAppStoreConnectPrivateKey(argParser);
        addCertificateKey(argParser);
        addWhatsNewOption(argParser);
    }

    static const flavorOptionName = 'flavor';
    static const exportOptionsPlistName = 'exportOptionsPlist';

    List<String> get _iosStages => _iosStageToRun;

    @override
    String get description =>
        'Deploys the Sharezone iOS app to the App Store';

    @override
    String get name => 'ios';

    @override
    Future<void> run() async {
        _throwIfFlavorIsNotSupportForDeployment();
        await throwIfCodemagicCliToolsAreNotInstalled();

        final whatsNew = argResults![whatsNewOptionName];
        stdout.writeln('Deploy iOS with the following options:');
    }

```

My problem is that the "--whats-new" command is null but I don't understand why. I can't reproduce it locally. On my main branch the logs are the following:

```
LAST_COMMIT_MESSAGE: Add temporary logs to the main branch
```

I tried it on a feature branch but it worked fine. I tried it on the `main` branch.

Related to #922

```
SHORT_LAST_COMMIT_MESSAGE: Add temporary logs to the main branch
```

I tried it on a feature branch but it worked on the `main` branch.

Related to #922

Deploy iOS with the following release notes
null

So you can see, the var
"SHORT_LAST_COMMIT_MESSAGE" has the changelog message but when coming to the Dart and my print statement, the "whatsNew" var is null. Why?

It seems that your `SHORT_LAST_COMMIT_MESSAGE` environment variable is not being passed correctly to your Dart script. In Dart, when you use `argResults!` [`whatsNewOptionName`] to get the value of a command line option, it doesn't automatically pull from environment variables — it only pulls from the actual command line arguments passed to the script.

To make your script use the `SHORT_LAST_COMMIT_MESSAGE` environment variable, you would have to explicitly pass it to the `sz deploy ios` command using shell variable expansion, like this:

yaml

 Copy code

```
sz deploy ios \  
  --stage alpha \  
  --whats-new="$SHORT_LAST_COMMIT_MESSAGE" \  
  --export-options-plist=$HOME/export_options.plist
```

Given that you're already doing this in your YAML file, it seems like the issue might be happening because the shell is not expanding the variable correctly inside the double quotes. Sometimes shells can behave strangely with variable expansion inside quotes.

As a first step to debugging this, try removing the quotes around the variable to see if that makes a difference:

```
sz deploy ios \  
  --stage alpha \  
  --whats-new=$SHORT_LAST_COMMIT_MESSAGE \  
  --export-options-plist=$HOME/export_options.plist
```

If that doesn't work, another thing to try is to use single quotes instead of double quotes:

yaml

 Copy code

```
sz deploy ios \  
  --stage alpha \  
  --whats-new='$SHORT_LAST_COMMIT_MESSAGE' \  
  --export-options-plist=$HOME/export_options.plist
```

However, given that you're passing a multiline string, removing quotes might not work properly. Therefore, I would recommend using a simple placeholder value to see if the argument is being passed at all:

yaml

 Copy code

```
sz deploy ios \  
  --stage alpha \  
  --whats-new="TEST_MESSAGE" \  
  --export-options-plist=$HOME/export_options.plist
```

If "TEST_MESSAGE" does get passed correctly, then the issue is likely with how the shell is handling the multiline `SHORT_LAST_COMMIT_MESSAGE` variable. In that case, you might try replacing newline characters with spaces or some other character to ensure the entire message gets passed as a single line, like so:

yaml

 Copy code

```
export SHORT_LAST_COMMIT_MESSAGE=$(echo $LAST_COMMIT_MESSAGE | tr '\n' ' ')
```

Try these suggestions one after the other to pinpoint where the problem is coming from. Once you've identified the issue, it should be easier to find a solution.

Okay. Good news. I'm able to reproduce it locally. I think it has something to do with multiline strings. When the string is multiline, it's null. Any ideas what to do next?

Yes, it seems like you're facing an issue with how the shell handles multiline strings. You have a few options for how to proceed:

1. Replace newline characters with spaces: Before passing the

`SHORT_LAST_COMMIT_MESSAGE` variable to the Dart script, you could replace all newline characters with spaces to make it a single-line string. You've already made this transformation to some extent in your shell script:

yaml

 Copy code

```
export LAST_COMMIT_MESSAGE=$(git log -1 --pretty=%B | sed 's/[<>]//g')
export SHORT_LAST_COMMIT_MESSAGE=$(echo $LAST_COMMIT_MESSAGE | tr '\n'
```

2. Use Base64 encoding and decoding: You can encode the multiline string to a single-line Base64 string and decode it in your Dart script. Here's how you can do it:

In your GitHub Action:

yaml

 Copy code

```
export SHORT_LAST_COMMIT_MESSAGE=$(echo "$LAST_COMMIT_MESSAGE" | base64
```

And in your Dart script, decode it:

dart

 Copy code

```
import 'dart:convert';

final whatsNew = argResults![whatsNewOptionName] as String?;
if (whatsNew != null) {
  final decodedWhatsNew = utf8.decode(base64.decode(whatsNew));
}
```

```
stdout.writeln('Deploy iOS with the following release notes:\n$decode
}
```

- 3. Pass the Message as a File:** If your release notes can be very large, you might hit shell command length limits. Another safe option is to write the message to a temporary file and then read it in your Dart program.

In your GitHub Action:

yaml

 Copy code

```
echo "$LAST_COMMIT_MESSAGE" > temp_release_notes.txt
```

Pass the filename to your Dart program:

yaml

 Copy code

```
sz deploy ios \
  --stage alpha \
  --whats-new=temp_release_notes.txt \
  --export-options-plist=$HOME/export_options.plist
```

And in your Dart program, read the contents of the file:

dart

 Copy code

```
import 'dart:io';

final whatsNew = argResults![whatsNewOptionName] as String?;
if (whatsNew != null) {
  final file = File(whatsNew);
  if (await file.exists()) {
    final whatsNewContent = await file.readAsString();
    stdout.writeln('Deploy iOS with the following release notes:\n$what
  } else {
    stdout.writeln('Deploy iOS with the following release notes:\n$what
  }
}
```

I recommend trying the first solution. If it doesn't work, then you can try the second or third solution as a more robust fix. Let me know how it goes!

```
sz deploy ios \  
  --stage alpha \  
  --whats-new "$SHORT_LAST_COMMIT_MESSAGE" \  
  --export-options-plist=$HOME/export_options.plist
```

This seems to work when I leave the equals symbol (=)

I'm glad to hear that it's working now! Yes, sometimes the presence or absence of certain characters such as the equals symbol (=) can affect how arguments are parsed in shell commands.

In bash and similar shells, you generally don't need the equals symbol when passing arguments to a script; space is used to separate the option name from its value. However, it's not uncommon to see the equals symbol used in this way, and many scripts are set up to accept arguments in either format.

Here's a quick recap of what worked:

yaml

 Copy code

```
sz deploy ios \  
  --stage alpha \  
  --whats-new "$SHORT_LAST_COMMIT_MESSAGE" \  
  --export-options-plist=$HOME/export_options.plist
```

You used double quotes around `$SHORT_LAST_COMMIT_MESSAGE` to preserve the multiline nature of the string when it is passed to the `sz deploy ios` command. This makes it so the entire multi-line string is treated as a single argument, which is then correctly captured by your Dart script.

Going forward, remember to always test with different kinds of inputs (including multiline strings) when working with script arguments to avoid such issues.

If you have any other questions, feel free to ask.

Can you write a short comment why we leave the = symbol that I can add to the action yaml?

Absolutely, here's a concise comment you can add to your GitHub Actions YAML file to explain why the equals symbol (=) is omitted:

yaml

 Copy code

```
# We omit the "=" symbol when passing the --whats-new argument to ensure that the multiline string stored in the $SHORT_LAST_COMMIT_MESSAGE variable is treated as a single argument. Including "=" can cause issues with argument parsing, leading to the multiline string being incorrectly handled.
```

This comment explains the reason behind omitting the equals symbol and highlights the fact that doing so is important for correctly handling multiline strings. Feel free to adjust the wording to better suit your documentation style!

Please start the comment with something like "Note that use '--whats-new \$SHORT_LAST_COMMIT_MESSAGE' instead of '--whats-new=\$SHORT_LAST_COMMIT_MESSAGE'

Absolutely, here's an updated version of the comment:

yaml

 Copy code

```
# Note that we use '--whats-new "$SHORT_LAST_COMMIT_MESSAGE"' instead of '--whats-new="$SHORT_LAST_COMMIT_MESSAGE"' to correctly pass the multiline string stored in the $SHORT_LAST_COMMIT_MESSAGE variable as a single argument.
```

```
# Including "=" can cause issues with argument parsing in the shell, potentially  
# leading to incorrect handling of the multiline string argument.
```

This comment begins with a direct reference to the format that you are using, and highlights the reason for choosing this approach over the alternative that includes "=" for argument assignment. It should clearly communicate the rationale behind this choice to anyone reading the code in the future.

Write a short pull description for this change and mainly explain what the issue was.

Absolutely, here is a brief description for your pull request that explains the change and the issue that necessitated it:

PR Description

Fix Multiline Release Notes Handling in GitHub Action

In our previous GitHub Action configuration, the `--what's-new` parameter was passed with an equals symbol (`=`), as in `--what's-new="$SHORT_LAST_COMMIT_MESSAGE"`. However, this approach encountered issues when the release notes contained a multiline string, causing the multiline string not to be passed correctly to the Dart CLI tool, resulting in a null value received in the Dart script.

To resolve this, we modified the syntax to `--what's-new "$SHORT_LAST_COMMIT_MESSAGE"`, omitting the `=`. This ensures that multiline strings are correctly passed as a single argument to the CLI tool, preserving the integrity of the release notes.

This update ensures that release notes for alpha versions generated from multiline commit messages are correctly handled and displayed.

Feel free to adjust the wording to better suit your project's style or to add any additional details you feel are necessary.